

Autoescuela Hermemehildo



I DAW -- Bases de Datos

Anxo Campos

Servando Hernández

Índice:

Supuesto:.....	3
Diagrama - CFM.....	5
Diagrama —Modelo Relacional.....	6
Tablas.....	7
Creates y Alters:.....	11
Inserts.....	16
Consultas.....	21
Updates.....	25
Deletes.....	26
Transacciones.....	27
Vistas.....	30
Funciones.....	32
Procedimiento.....	36
Triggers.....	39
Backup.....	43
Explicación de los parámetros:.....	43

Supuesto:

La autoescuela mantiene un registro único de personas que recoge los datos de identificación y contacto (nombre, apellidos, DNI, fecha de nacimiento, dirección y teléfono). A partir de ese registro se gestionan dos perfiles principales:

Empleados, que representan al personal de la autoescuela y se vinculan 1:1 con una persona. Entre empleados puede existir una relación jerárquica mediante un responsable (jefe). Además, un empleado puede disponer de varios teléfonos de empresa.

Estudiantes, que representan al alumnado matriculado y se vinculan 1:1 con una persona.

Vehículos de prácticas

La autoescuela dispone de varios vehículos, cada uno identificado por su matrícula y con información opcional (marca, modelo y color). Cada estudiante tiene asignado un único vehículo para la realización de las clases prácticas.

Asignación de responsables

Cada estudiante queda vinculado a un empleado responsable (por ejemplo, profesor de referencia o tutor), que se utiliza como referencia organizativa dentro de la autoescuela.

Prácticas (clases prácticas)

Para poder conocer cuántas prácticas realiza cada estudiante, se registra cada sesión de práctica indicando:

el estudiante,

la fecha,

la duración real (en minutos), y opcionalmente:

el instructor (empleado) que impartió la práctica,

el vehículo utilizado,

observaciones.

Pruebas y exámenes

La autoescuela gestiona distintos tipos de evaluaciones:

Tests internos: se definen por un número de preguntas, un tiempo máximo y un tema (con dispositivo opcional).

Exámenes teóricos: se definen por una ubicación, un número de preguntas, un tiempo máximo y una categoría de permiso.

Exámenes prácticos: se definen por una ubicación, una duración aproximada y una categoría de permiso.

Los exámenes teóricos y prácticos se clasifican por la categoría del permiso (A, B, C, D, E o T).

Criterio de tiempos

Todas las duraciones y tiempos máximos se registran como valores numéricos en minutos, para evitar ambigüedades y facilitar validaciones y cálculos (por ejemplo, sumas y medias).

Resultados

Cada vez que un estudiante realiza una prueba, se registra un resultado asociado al estudiante y a la prueba correspondiente. En cada resultado se almacena:

la fecha,

la duración real (en minutos),

y el indicador de superación cuando aplica.

En concreto:

En tests, se registra si la prueba fue superada y el número de fallos.

En examen teórico, se registra la fecha y el número de aciertos del estudiante (además de la duración real).

En examen práctico, se registra si fue superado, la duración real y el número de faltas.

Examinador externo

En el examen práctico se identifica:

el profesor, que siempre es un empleado (acompañante o responsable de la preparación), y

el examinador, que se registra como persona y puede ser externo a la autoescuela (no se presupone que sea empleado).

Integridad y restricciones

El modelo aplica reglas de integridad para garantizar coherencia, por ejemplo:

DNI único por persona.

Valores positivos en cantidades de preguntas y duraciones.

En examen práctico, el examinador y el profesor no pueden ser la misma persona.

Las prácticas y resultados quedan vinculados a estudiantes existentes, y las relaciones con empleados/vehículos se controlan mediante referencias y reglas de borrado coherentes.

Diagrama - CFM

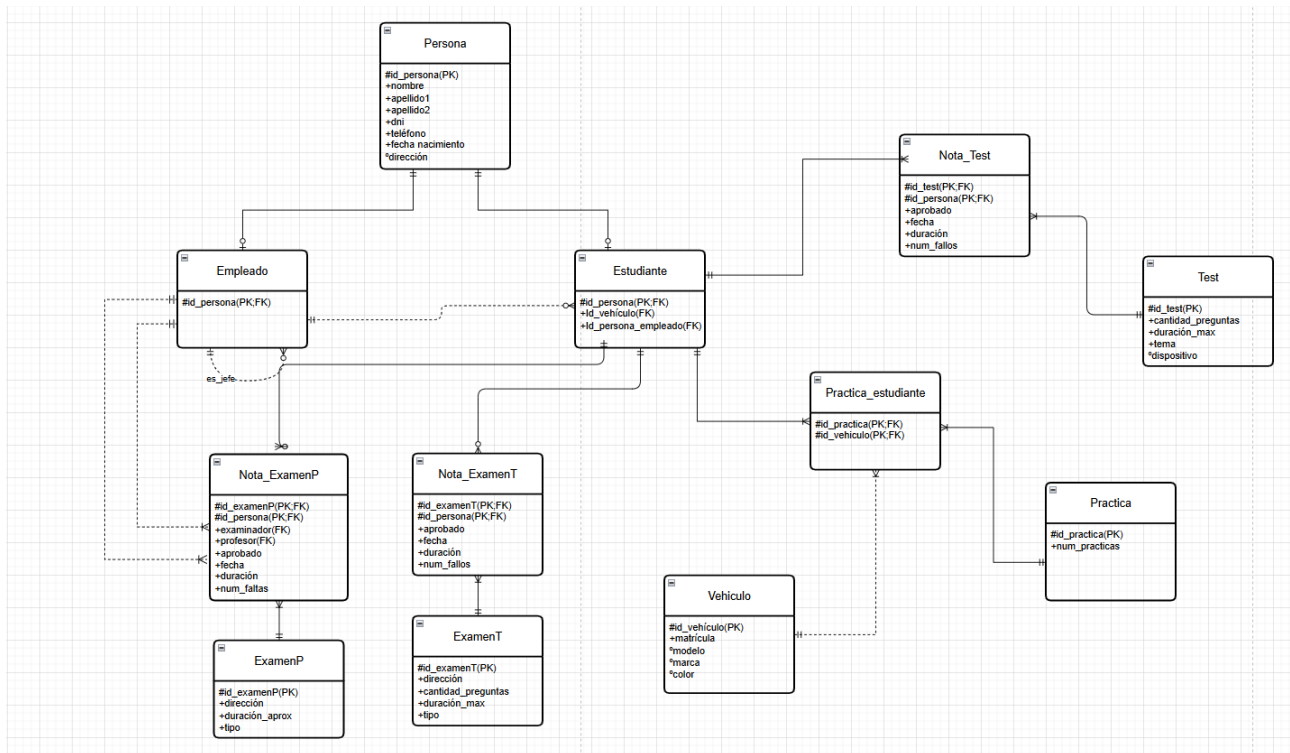
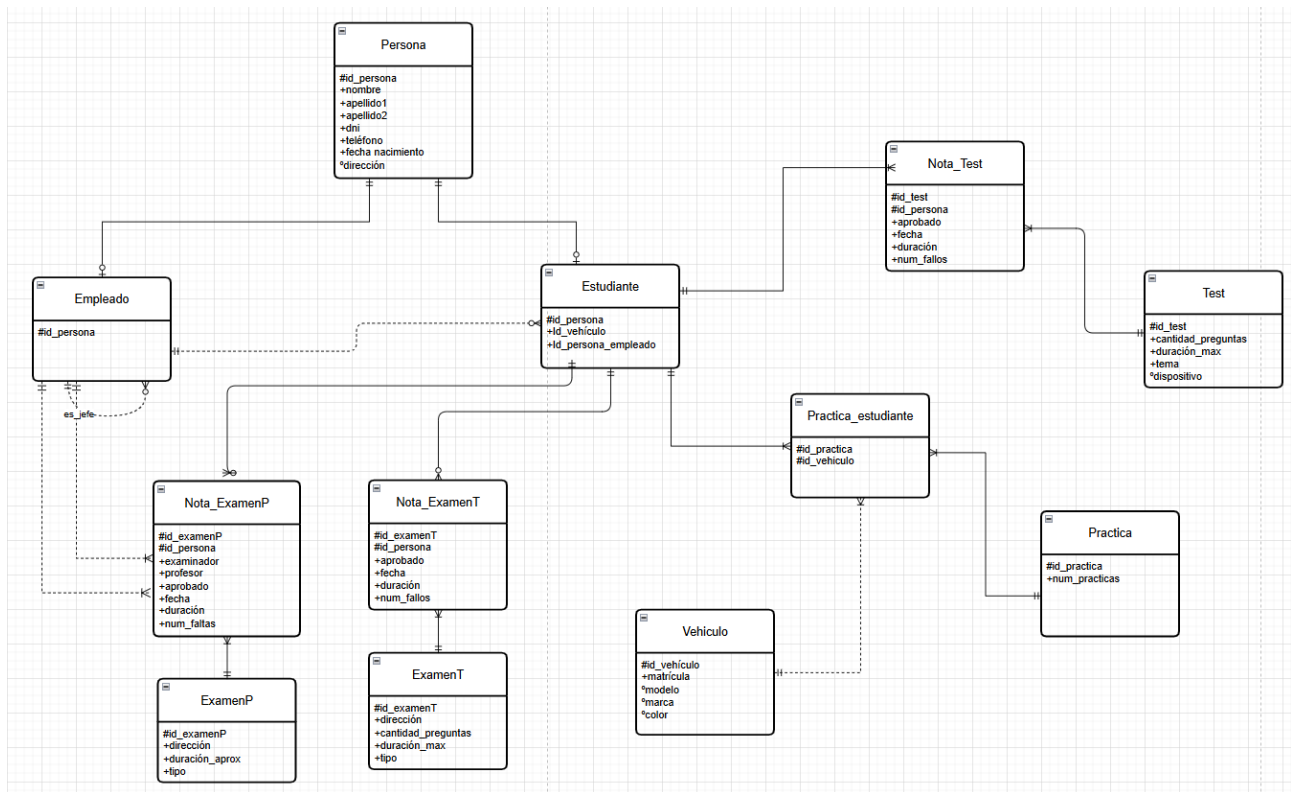


Diagrama - Modelo Relacional



Tablas

Tests

Nombre de la columna	Tipo de dato	Claves	NULL	Observaciones
idtest	SERIAL	PK	NOT NULL	autoincremental
cantidad_preguntas	INT		NOT NULL	> 0 (restricción CHECK)
duracion_max	VARCHAR(10)		NOT NULL	Minutos, > 0(CHECK)
tema	TEXT		NOT NULL	
dispositivo	VARCHAR(50)		NULL	

Vehículos

Nombre de la columna	Tipo de dato	Claves	NULL	Observaciones
idvehiculo	SERIAL	PK	NOT NULL	autoincremental
matricula	VARCHAR(7)	UQ	NOT NULL	única
modelo	VARCHAR(10)		NULL	
marca	VARCHAR(10)		NULL	
color	VARCHAR(30)		NULL	

ExámenesP

Nombre de la columna	Tipo de dato	Claves	NULL	Observaciones
idexamenP	SERIAL	PK	NOT NULL	autoincremental
direccion	TEXT		NOT NULL	
duracion_aprox	VARCHAR(10)		NOT NULL	Minutos, > 0 (CHECK)
tipo	tipo_enum		NOT NULL	valores: B,A,C,D,E,T

ExámenesT

Nombre de la columna	Tipo de dato	Claves	NULL	Observaciones
idexamenT	SERIAL	PK	NOT NULL	autoincremental
direccion	TEXT		NOT NULL	
cantidad_preguntas	INT		NOT NULL	> 0 (CHECK)
duracion_aprox	VARCHAR(10)		NOT NULL	Minutos, > 0 (CHECK)
tipo	tipo_enum		NOT NULL	valores: B,A,C,D,E,T

Persona

Nombre de la columna	Tipo de dato	Claves	NULL	Observaciones
idpersona	SERIAL	PK	NOT NULL	autoincremental
nombre	VARCHAR(20)		NOT NULL	
apellido1	VARCHAR(30)		NOT NULL	
apellido2	VARCHAR(30)		NULL	
dni	VARCHAR(9)	UNIQUE	NOT NULL	único
fecha_nac	DATE		NOT NULL	
direccion	TEXT		NULL	
telefono	VARCHAR(12)		NOT NULL	

Empleados

Nombre de la columna	Tipo de dato	Claves	NULL	Observaciones
idempleado	SERIAL	PK	NOT NULL	autoincremental
nombre	VARCHAR(20)		NOT NULL	
apellido1	VARCHAR(30)		NOT NULL	
dni	VARCHAR(9)	UNIQUE	NOT NULL	único
fecha_nac	DATE		NOT NULL	
direccion	TEXT		NULL	
telefono	TEXT		NOT NULL	varios teléfonos
idjefe	INT	FK	NULL	relación reflexiva con empleados

Estudiantes

Nombre de la columna	Tipo de dato	Claves	NULL	Observaciones
idestudiante	SERIAL	PK	NOT NULL	autoincremental
nombre	VARCHAR(20)		NOT NULL	
apellido1	VARCHAR(30)		NOT NULL	
apellido2	VARCHAR(30)		NULL	
dni	VARCHAR(9)	UNIQUE	NOT NULL	único
fecha_nac	DATE		NOT NULL	
direccion	TEXT		NULL	
telefono	VARCHAR(12)		NOT NULL	
idvehiculo	INT	FK	NOT NULL	referencia a vehiculos
id_persona_empleado	INT	FK	NOT NULL	referencia a empleados

Notas Tests

Nombre de la columna	Tipo de dato	Claves	NULL	Observaciones
idtest	INT	PK / FK	NOT NULL	referencia a tests
idestudiante	INT	PK / FK	NOT NULL	referencia a estudiantes
aprobado	BOOLEAN		NOT NULL	
fecha	DATE		NOT NULL	
duracion	VARCHAR(10)		NOT NULL	
num_fallos	INT		NOT NULL	
(PK compuesta: idtest + idestudiante)				

Notas ExamenP

Nombre de la columna	Tipo de dato	Claves	NULL	Observaciones
idexamenP	INT	PK / FK	NOT NULL	referencia a examenesP
idestudiante	INT	PK / FK	NOT NULL	referencia a estudiantes
examinador	INT	FK	NOT NULL	referencia a empleados
profesor	INT	FK	NOT NULL	referencia a empleados
duracion	VARCHAR(10)		NOT NULL	
aprobado	BOOLEAN		NOT NULL	
fecha	DATE		NOT NULL	
num_fallos	INT		NOT NULL	
(check)				Examinador <> profesor
(PK compuesta: idexamenP + idestudiante)				

Notas ExamenT

Nombre de la columna	Tipo de dato	Claves	NULL	Observaciones
idexamenT	INT	PK / FK	NOT NULL	referencia a examenesT
idestudiante	INT	PK / FK	NOT NULL	referencia a estudiantes
duracion	VARCHAR(10)		NOT NULL	
aprobado	BOOLEAN		NOT NULL	
fecha	DATE		NOT NULL	
num_fallos	INT		NOT NULL	
(PK compuesta: idexamenT + idestudiante)				

Creates y Alters:

```
DROP TABLE IF EXISTS notas_examenT CASCADE;
DROP TABLE IF EXISTS notas_examenP CASCADE;
DROP TABLE IF EXISTS notas_tests CASCADE;
DROP TABLE IF EXISTS practicas CASCADE;
DROP TABLE IF EXISTS estudiantes CASCADE;
DROP TABLE IF EXISTS empleados CASCADE;
DROP TABLE IF EXISTS personas CASCADE;
DROP TABLE IF EXISTS examenesT CASCADE;
DROP TABLE IF EXISTS examenesP CASCADE;
DROP TABLE IF EXISTS vehiculos CASCADE;
DROP TABLE IF EXISTS tests CASCADE;
DROP TYPE IF EXISTS tipo_enum;

-- Tipos
CREATE TYPE tipo_enum AS ENUM ('B','A','C','D','E','T');

-- Entidades base
CREATE TABLE personas (
    idpersona SERIAL PRIMARY KEY,
    nombre VARCHAR(50) NOT NULL,
    apellido1 VARCHAR(60) NOT NULL,
    apellido2 VARCHAR(60),
    dni VARCHAR(9) NOT NULL UNIQUE,
    fecha_nac DATE NOT NULL,
    direccion TEXT,
    telefono TEXT NOT NULL
);

-- Empleado
CREATE TABLE empleados (
    telefonos_empresa TEXT[] DEFAULT ARRAY[]::TEXT[]
) INHERITS (personas);

-- Vehículos
CREATE TABLE vehiculos (
    idvehiculo SERIAL PRIMARY KEY,
    matricula VARCHAR(7) NOT NULL UNIQUE,
    modelo VARCHAR(30),
    marca VARCHAR(30),
    color VARCHAR(30)
);
```

```

-- Estudiante
CREATE TABLE estudiantes (
    idvehiculo INT NOT NULL REFERENCES vehiculos(idvehiculo) ON DELETE RESTRICT
) INHERITS (personas);

-- Definición de tests
CREATE TABLE tests (
    idtest SERIAL PRIMARY KEY,
    cantidad_preguntas INT NOT NULL CHECK (cantidad_preguntas > 0),
    duracion_max_minutos INT NOT NULL CHECK (duracion_max_minutos > 0),
    tema TEXT NOT NULL,
    dispositivo VARCHAR(50)
);

-- Definición de exámenes (práctico / teórico)
CREATE TABLE examenesP (
    idexamenP SERIAL PRIMARY KEY,
    direccion TEXT NOT NULL,
    duracion_aprox_minutos INT NOT NULL CHECK (duracion_aprox_minutos > 0),
    tipo tipo_enum NOT NULL
);

CREATE TABLE examenesT (
    idexamenT SERIAL PRIMARY KEY,
    direccion TEXT NOT NULL,
    cantidad_preguntas INT NOT NULL CHECK (cantidad_preguntas > 0),
    duracion_max_minutos INT NOT NULL CHECK (duracion_max_minutos > 0),
    tipo tipo_enum NOT NULL
);

-- Prácticas:
CREATE TABLE practicas (
    idpractica SERIAL PRIMARY KEY,
    idestudiante INT REFERENCES personas(idpersona) ON DELETE CASCADE,
    idempleado_instructor INT REFERENCES personas(idpersona) ON DELETE SET NULL,
    idvehiculo INT REFERENCES vehiculos(idvehiculo) ON DELETE SET NULL,
    fecha DATE NOT NULL,
    duracion_minutos INT NOT NULL CHECK (duracion_minutos > 0),
    observaciones TEXT
);

```

```

-- Resultados (intentos) de tests
CREATE TABLE notas_tests (
    idtest INT REFERENCES tests(idtest) ON DELETE CASCADE,
    idestudiante INT REFERENCES personas(idpersona) ON DELETE CASCADE,
    aprobado BOOLEAN NOT NULL,
    fecha DATE NOT NULL,
    duracion_minutos INT NOT NULL CHECK (duracion_minutos > 0),
    num_fallos INT NOT NULL CHECK (num_fallos >= 0),
    PRIMARY KEY (idtest, idestudiante, fecha)
);

-- Resultados(intentos) de examen práctico
-- Examinador es una persona , profesor es empleado.
CREATE TABLE notas_examenP (
    idexamenP INT REFERENCES examenesP(idexamenP) ON DELETE CASCADE,
    idestudiante INT REFERENCES personas(idpersona) ON DELETE CASCADE,
    idexaminador INT NOT NULL REFERENCES personas(idpersona) ON DELETE RESTRICT,
    idprofesor INT NOT NULL REFERENCES personas(idpersona) ON DELETE RESTRICT,
    duracion_minutos INT NOT NULL CHECK (duracion_minutos > 0),
    aprobado BOOLEAN NOT NULL,
    fecha DATE NOT NULL,
    num_faltas INT NOT NULL CHECK (num_faltas >= 0),
    PRIMARY KEY (idexamenP, idestudiante, fecha)
);

-- Resultados (intentos) de examen teórico:
-- Se guarda fecha y "num_aciertos"
CREATE TABLE notas_examenT (
    idexamenT INT REFERENCES examenesT(idexamenT) ON DELETE CASCADE,
    idestudiante INT REFERENCES personas(idpersona) ON DELETE CASCADE,
    fecha DATE NOT NULL,
    duracion_minutos INT NOT NULL CHECK (duracion_minutos > 0),
    num_aciertos INT NOT NULL CHECK (num_aciertos >= 0),
    PRIMARY KEY (idexamenT, idestudiante, fecha)
);

-- Índices útiles
CREATE INDEX idx_practicas_estudiante ON practicas(idestudiante);
CREATE INDEX idx_notasT_estudiante ON notas_examenT(idestudiante);
CREATE INDEX idx_notasP_estudiante ON notas_examenP(idestudiante);
CREATE INDEX idx_notas_tests_estudiante ON notas_tests(idestudiante);

```

```

ALTER TABLE empleados
ADD COLUMN idjefe INT REFERENCES personas(idpersona);
/*Puede tener mas de 1 telefono (telefonos de empresa)*/
ALTER TABLE empleados
ALTER COLUMN telefonos_empresa TYPE TEXT[]
/*convertimos todos los datos existentes a TEXT[]*/
USING ARRAY[telefonos_empresa];
/*hace que los dnis introducidos sean unicos */
ALTER TABLE personas
ADD CONSTRAINT dni_unico UNIQUE (dni);

ALTER TABLE estudiantes
ADD CONSTRAINT dni_unico_estudiantes UNIQUE (dni);

ALTER TABLE empleados
ADD CONSTRAINT dni_unico_empleados UNIQUE (dni);
/*chequea que el id del examinador nosea el mismo que el profesor*/
ALTER TABLE notas_examenP
ADD CONSTRAINT chk_examinador_profesor_distintos
CHECK (idexaminador <> idprofesor);

```

Inserts

Vehiculos:

```
INSERT INTO vehiculos(matricula, modelo, marca, color) VALUES
('1111AAA', 'GTI', 'Nissan', 'Negro'),
('2222BBB', 'TDI', 'Volkswagen', 'Azul'),
('3333CCC', 'Fiesta', 'Ford', 'Blanco'),
('4444DDD', 'Getz', 'Hyundai', 'Gris'),
('5555EEE', 'Corolla', 'Toyota', 'Rojo'),
('6666FFF', 'Ibiza', 'SEAT', 'Blanco'),
('7777GGG', 'Clio', 'Renault', 'Gris'),
('8888HHH', 'Corsa', 'Opel', 'Azul'),
('9999JJJ', 'Polo', 'Volkswagen', 'Negro'),
('0000KKK', '208', 'Peugeot', 'Rojo'),
('1212LLL', 'Focus', 'Ford', 'Azul'),
('3434MMM', 'Astra', 'Opel', 'Gris'),
('5656NNN', 'i20', 'Hyundai', 'Blanco'),
('7878PPP', 'Yaris', 'Toyota', 'Negro'),
('9090RRR', 'Golf', 'Volkswagen', 'Gris');
```

Empleados:

```
INSERT INTO personas(nombre, apellido1, apellido2, dni, fecha_nac, direccion, telefono) VALUES
('Anxo', 'Curras', NULL, '12345678X', '2000-11-10', 'Calle Calvar 99', '123 456 789'),
('Angel', 'Mendez', NULL, '87654321Z', '1999-07-11', 'Calle Pilar 87', '222 222 222'),
('Xurxo', 'Fisterra', 'Sotelo', '24681357R', '1998-02-12', 'Calle Santarosa 14', '333 333 333'),
('Jorge', 'Lago', 'Cordeiro', '78945612T', '1996-04-13', 'Calle Eduardo Chaves 69', '444 444 444'),
('Miguel', 'Chaves', 'Gonzales', '32165498Y', '1997-09-14', 'Calle Piñeiro 3', '555 555 555'),
('Pepe', 'Del Rio', NULL, '15975346H', '1988-01-15', 'Calle Calvar 1', '666 666 666'),
('Manuel', 'Magallanes', NULL, '46829715L', '1985-03-16', 'Calle Piñeiro 33', '777 777 777'),
('Xian', 'Pazo', 'Quente', '90753124M', '1990-05-17', 'Calle Santarosa 66', '888 888 888'),
('Juan', 'Soto', 'Silva', '58241976K', '1989-06-18', 'Calle Eduardo Chaves 9', '999 999 999'),
('Marta', 'Reina', NULL, '73486219P', '1991-08-19', 'Calle Pilar 87', '111 111 111'),
('Noa', 'Varela', NULL, '11223344A', '1993-10-20', 'Rua do Sol 7', '111 222 333'),
('Iria', 'Dominguez', 'Lopez', '22334455B', '1992-12-21', 'Avd Galicia 12', '222 333 444'),
('Hugo', 'Pereira', NULL, '33445566C', '1987-11-22', 'Rua Nova 5', '333 444 555'),
('Lara', 'Suarez', 'Mato', '44556677D', '1994-02-23', 'Calle Palmas 3', '444 555 666'),
('Brais', 'Molina', NULL, '55667788E', '1986-07-24', 'Rua San Xoan 10', '555 666 777');
```

```
INSERT INTO empleados(idpersona, nombre, apellido1, apellido2, dni, fecha_nac, direccion, telefono, idjefe, telefonos_empresa) VALUES
(1, 'Anxo', 'Curras', NULL, '12345678X', '2000-11-10', 'Calle Calvar 99', '123 456 789', NULL, ARRAY['981 111 111']),
(2, 'Angel', 'Mendez', NULL, '87654321Z', '1999-07-11', 'Calle Pilar 87', '222 222 222', NULL, ARRAY['981 111 112', '981 111 212']),
(3, 'Xurxo', 'Fisterra', 'Sotelo', '24681357R', '1998-02-12', 'Calle Santarosa 14', '333 333 333', NULL, ARRAY['981 111 113']),
(4, 'Jorge', 'Lago', 'Cordeiro', '78945612T', '1996-04-13', 'Calle Eduardo Chaves 69', '444 444 444', NULL, ARRAY['981 111 114']),
(5, 'Miguel', 'Chaves', 'Gonzales', '32165498Y', '1997-09-14', 'Calle Piñeiro 3', '555 555 555', NULL, ARRAY['981 111 115']),
(6, 'Pepe', 'Del Rio', NULL, '15975346H', '1988-01-15', 'Calle Calvar 1', '666 666 666', NULL, ARRAY['981 111 116']),
(7, 'Manuel', 'Magallanes', NULL, '46829715L', '1985-03-16', 'Calle Piñeiro 33', '777 777 777', NULL, ARRAY['981 111 117']),
(8, 'Xian', 'Pazo', 'Quente', '90753124M', '1990-05-17', 'Calle Santarosa 66', '888 888 888', NULL, ARRAY['981 111 118']),
(9, 'Juan', 'Soto', 'Silva', '58241976K', '1989-06-18', 'Calle Eduardo Chaves 9', '999 999 999', NULL, ARRAY['981 111 119']),
(10, 'Marta', 'Reina', NULL, '73486219P', '1991-08-19', 'Calle Pilar 87', '111 111 111', NULL, ARRAY['981 111 120']),
(11, 'Noa', 'Varela', NULL, '11223344A', '1993-10-20', 'Rua do Sol 7', '111 222 333', NULL, ARRAY['981 111 121']),
(12, 'Iria', 'Dominguez', 'Lopez', '22334455B', '1992-12-21', 'Avd Galicia 12', '222 333 444', NULL, ARRAY['981 111 122']),
(13, 'Hugo', 'Pereira', NULL, '33445566C', '1987-11-22', 'Rua Nova 5', '333 444 555', NULL, ARRAY['981 111 123']),
(14, 'Lara', 'Suarez', 'Mato', '44556677D', '1994-02-23', 'Calle Palmas 3', '444 555 666', NULL, ARRAY['981 111 124']),
(15, 'Brais', 'Molina', NULL, '55667788E', '1986-07-24', 'Rua San Xoan 10', '555 666 777', NULL, ARRAY['981 111 125']);
```

Estudiantes:

```
INSERT INTO personas(nombre, apellido1, apellido2, dni, fecha_nac, direccion, telefono) VALUES
('Sebas','Vazques','Rios','11112222T','2008-11-15','Calle Calvar 1','666 666 666'),
('Aron','Martinez',NULL,'33334444R','2008-11-16','Calle Pilar 8771','777 777 777'),
('Marcos','Antepazo','Ferradas','55556666Y','2008-11-17','Calle Santarosa 87','888 888 888'),
('Alejandro','Perez',NULL,'77778888Q','2008-11-18','Calle Eduardo Chaves 13','999 999 999'),
('Javi','Pucelas','Pereira','99998765S','2008-11-19','Calle Piñeiro 45','111 999 555'),
('Nerea','Santos',NULL,'66778899F','2007-01-05','Rua do Regueiro 15','600 000 001'),
('Adrian','Gomez','Paz','77889900G','2007-02-06','Rua do Regueiro 15','600 000 002'),
('Lucía','Rodríguez',NULL,'88990011I','2007-03-07','Rua do Regueiro 15','600 000 003'),
('Pablo','Alvarez','Seoane','99001122J','2007-04-08','Rua do Regueiro 15','600 000 004'),
('Sara','Prieto',NULL,'10101010K','2007-05-09','Rua do Regueiro 15','600 000 005'),
('Diego','Vazquez',NULL,'20202020L','2007-06-10','Calle Calvar 22','600 000 006'),
('Ainhoa','Crespo','Lage','30303030M','2007-07-11','Calle Calvar 23','600 000 007'),
('Izan','Nogueira',NULL,'40404040N','2007-08-12','Calle Calvar 24','600 000 008'),
('Carla','Souto','Pita','50505050O','2007-09-13','Calle Calvar 25','600 000 009'),
('Eric','Vidal',NULL,'60606060R','2007-10-14','Calle Calvar 26','600 000 010');
```

```
INSERT INTO estudiantes(idpersona,nombre, apellido1, apellido2, dni, fecha_nac, direccion, telefono, idvehiculo) VALUES
(16,'Sebas','Vazques','Rios','11112222T','2008-11-15','Calle Calvar 1','666 666 666', 1),
(17,'Aron','Martinez',NULL,'33334444R','2008-11-16','Calle Pilar 8771','777 777 777', 2),
(18,'Marcos','Antepazo','Ferradas','55556666Y','2008-11-17','Calle Santarosa 87','888 888 888', 3),
(19,'Alejandro','Perez',NULL,'77778888Q','2008-11-18','Calle Eduardo Chaves 13','999 999 999', 4),
(20,'Javi','Pucelas','Pereira','99998765S','2008-11-19','Calle Piñeiro 45','111 999 555', 5),
(21,'Nerea','Santos',NULL,'66778899F','2007-01-05','Rua do Regueiro 15','600 000 001', 6),
(22,'Adrian','Gomez','Paz','77889900G','2007-02-06','Rua do Regueiro 15','600 000 002', 7),
(23,'Lucía','Rodríguez',NULL,'88990011I','2007-03-07','Rua do Regueiro 15','600 000 003', 8),
(24,'Pablo','Alvarez','Seoane','99001122J','2007-04-08','Rua do Regueiro 15','600 000 004', 9),
(25,'Sara','Prieto',NULL,'10101010K','2007-05-09','Rua do Regueiro 15','600 000 005', 10),
(26,'Diego','Vazquez',NULL,'20202020L','2007-06-10','Calle Calvar 22','600 000 006',11),
(27,'Ainhoa','Crespo','Lage','30303030M','2007-07-11','Calle Calvar 23','600 000 007', 12),
(28,'Izan','Nogueira',NULL,'40404040N','2007-08-12','Calle Calvar 24','600 000 008',13),
(29,'Carla','Souto','Pita','50505050O','2007-09-13','Calle Calvar 25','600 000 009', 14),
(30,'Eric','Vidal',NULL,'60606060R','2007-10-14','Calle Calvar 26','600 000 010', 15);
```

Tests:

```
INSERT INTO tests(cantidad_preguntas, duracion_max_minutos, tema, dispositivo) VALUES
(25,25,'Luces','Movil'),
(30,20,'Carrocería','Ordenador'),
(17,30,'Actuación en accidente','Tablet_auto'),
(10,12,'Emergencias','Movil'),
(22,27,'Mantenimiento','PC'),
(20,20,'Señales','Movil'),
(15,15,'Prioridad de paso','Tablet_auto'),
(18,22,'Velocidades','Ordenador'),
(12,10,'Documentación','Movil'),
(28,30,'Conducción eficiente','PC'),
(16,18,'Alcohol y drogas','Movil'),
(24,25,'Adelantamientos','Tablet_auto'),
(14,14,'Intersecciones','Ordenador'),
(26,28,'Túneles','PC'),
(19,21,'Meteorología','Movil');
```


Exámenes Teóricos y Practicos:

```
INSERT INTO examenesP(direccion, duracion_aprox_minutos, tipo) VALUES
('García Barbon',30,'B'),
('Sanjurjo Badía',26,'D'),
('Avd de Galicia',27,'A'),
('Calle Palmas',35,'T'),
('Rua San Xoan',31,'C'),
('Rua Venezuela',28,'B'),
('Gran Vía',32,'B'),
('Rua Urzaiz',29,'A'),
('Plaza España',34,'C'),
('Travesía de Vigo',30,'D'),
('Rua Aragón',27,'E'),
('Rua Barcelona',33,'B'),
('Rua Zamora',25,'A'),
('Rua Bolivia',36,'T'),
('Rua Coruña',28,'C');
```

-- 7) Exámenes teóricos (15)

```
INSERT INTO examenesT(direccion, cantidad_preguntas, duracion_max_minutos, tipo) VALUES
('Rua do Regueiro 15',30,30,'A'),
('Rua do Regueiro 15',30,40,'B'),
('Rua do Regueiro 15',30,25,'T'),
('Rua do Regueiro 15',30,21,'C'),
('Rua do Regueiro 15',30,17,'D'),
('Rua do Regueiro 15',25,25,'B'),
('Rua do Regueiro 15',20,20,'A'),
('Rua do Regueiro 15',35,35,'C'),
('Rua do Regueiro 15',40,45,'D'),
('Rua do Regueiro 15',15,15,'E'),
('Rua do Regueiro 15',10,12,'T'),
('Rua do Regueiro 15',28,30,'B'),
('Rua do Regueiro 15',26,28,'A'),
('Rua do Regueiro 15',32,35,'C'),
('Rua do Regueiro 15',22,25,'D');
```

Prácticas:

```
INSERT INTO practicas(idestudiante, idempleado_instructor, idvehiculo, fecha, duracion_minutos, observaciones) VALUES
(16,1, 1,'2025-12-01',60,'Circuito básico'),
(16,2, 2,'2025-12-02',55,'Arranque en pendiente'),
(18,3, 3,'2025-12-03',50,'Rotondas'),
(19,4, 4,'2025-12-04',65,'Incorporaciones'),
(21,5, 5,'2025-12-05',45,'Señalización'),
(21,6, 6,'2025-12-06',60,'Maniobras'),
(22,7, 7,'2025-12-07',70,'Autovía'),
(23,8, 8,'2025-12-08',55,'Ciudad'),
(25,9, 9,'2025-12-09',50,'Estacionamiento'),
(25,10,10,'2025-12-10',65,'Conducción nocturna'),
(26,11,11,'2025-12-11',60,'Glorietas'),
(27,12,12,'2025-12-12',45,'Cambio de carril'),
(28,13,13,'2025-12-13',75,'Autopista'),
(29,14,14,'2025-12-14',50,'Cruces'),
(30,15,15,'2025-12-15',55,'Repaso general');
```

Notas tests:

```
-- 9) Notas de tests (15)
-- Usamos (idtest = 1-15) y (idestudiante = 16-30) 1 a 1, fechas distintas.
INSERT INTO notas_tests(idtest, idestudiante, aprobado, fecha, duracion_minutos, num_fallos) VALUES
(1, 16, TRUE, '2025-12-10', 20, 1),
(2, 17, TRUE, '2025-12-11', 18, 0),
(3, 18, FALSE, '2025-12-12', 25, 7),
(4, 19, FALSE, '2025-12-13', 15, 8),
(5, 20, TRUE, '2025-12-14', 30, 2),
(6, 21, TRUE, '2025-12-15', 19, 1),
(7, 22, FALSE, '2025-12-16', 14, 5),
(8, 23, TRUE, '2025-12-17', 21, 2),
(9, 24, TRUE, '2025-12-18', 10, 0),
(10,25, FALSE, '2025-12-19', 29, 6),
(11,26, TRUE, '2025-12-20', 17, 1),
(12,27, TRUE, '2025-12-21', 23, 2),
(13,28, FALSE, '2025-12-22', 13, 4),
(14,29, TRUE, '2025-12-23', 26, 2),
(15,30, TRUE, '2025-12-24', 20, 1);
```

Notas exámenes:

INSERT INTO notas_examenP **VALUES**

```
(1,16,11,1,26,TRUE,'2025-12-10',1),
(2,17,12,2,28,TRUE,'2025-12-11',0),
(3,18,13,3,37,FALSE,'2025-12-12',7),
(4,19,14,4,35,FALSE,'2025-12-13',8),
(5,20,15,5,27,TRUE,'2025-12-14',2),
(6,21,11,6,30,TRUE,'2025-12-15',1),
(7,22,12,7,32,FALSE,'2025-12-16',4),
(8,23,13,8,29,TRUE,'2025-12-17',2),
(9,24,14,9,34,TRUE,'2025-12-18',1),
(10,25,15,10,30,FALSE,'2025-12-19',5),
(11,26,11,1,27,TRUE,'2025-12-20',2),
(12,27,12,2,33,TRUE,'2025-12-21',1),
(13,28,13,3,25,FALSE,'2025-12-22',6),
(14,29,14,4,36,TRUE,'2025-12-23',2),
(15,30,15,5,28,TRUE,'2025-12-24',1);
```

-- 11) Notas de examen teórico (15)

INSERT INTO notas_examenT(idexamenT, idestudiante, fecha, duracion_minutos, num_aciertos) **VALUES**

```
(1, 16, '2025-12-10', 26, 28),
(2, 17, '2025-12-11', 28, 30),
(3, 18, '2025-12-12', 25, 18),
(4, 19, '2025-12-13', 21, 16),
(5, 20, '2025-12-14', 17, 27),
(6, 21, '2025-12-15', 20, 22),
(7, 22, '2025-12-16', 19, 15),
(8, 23, '2025-12-17', 24, 30),
(9, 24, '2025-12-18', 30, 33),
(10,25, '2025-12-19', 15, 10),
(11,26, '2025-12-20', 12, 9),
(12,27, '2025-12-21', 28, 26),
(13,28, '2025-12-22', 29, 20),
(14,29, '2025-12-23', 35, 30),
(15,30, '2025-12-24', 25, 21);
```

Consultas

1.- Joins que consulten la herencia

Mostrar los estudiantes con sus datos personales + vehículo + número de prácticas

```
1 /*1. Mostrar los estudiantes con sus datos personales + vehículo + número de prácticas.*/
2 SELECT e.nombre,e.apellido1,COALESCE(e.apellido2, '') AS apellido2,v.marca,v.modelo,COUNT(pr.idpractica) AS total_practicas
3 FROM estudiantes e
4 JOIN vehiculos v ON e.idvehiculo = v.idvehiculo
5 LEFT JOIN practicas pr ON e.idpersona = pr.idestudiante
6 GROUP BY e.nombre, e.apellido1, e.apellido2, v.marca, v.modelo
7 ORDER BY total_practicas DESC;
8
```

Data Output Messages Notifications

Showing rows: 1 to 15

	nombre character varying (50)	apellido1 character varying (60)	apellido2 character varying	marca character varying (30)	modelo character varying (30)	total_practicas bigint
1	Sebas	Vazques	Rios	Nissan	GTI	2
2	Sara	Prieto		Peugeot	208	2
3	Nerea	Santos		SEAT	Ibiza	2
4	Izan	Nogueira		Hyundai	i20	1
5	Marcos	Antepazo	Ferradas	Ford	Fiesta	1
6	Carla	Souto	Pita	Toyota	Yaris	1
7	Eric	Vidal		Volkswagen	Golf	1
8	Lucia	Rodriguez		Opel	Corsa	1
9	Diego	Vazquez		Ford	Focus	1
10	Alejandro	Perez		Hyundai	Getz	1
11	Ainhoa	Crespo	Lage	Opel	Astra	1
12	Adrian	Gomez	Paz	Renault	Clio	1
13	Aron	Martinez		Volkswagen	TDI	0
14	Pablo	Alvarez	Seoane	Volkswagen	Polo	0
15	Javi	Pucelas	Pereira	Toyota	Corolla	0

2.- Al menos 4 joins internos

2. Estudiantes con prácticas, test y exámenes prácticos aprobados. -> Muestra el número de prácticas, tests aprobados y exámenes prácticos aprobados por cada estudiante.

```
10 SELECT p.nombre, p.apellido1, COUNT(DISTINCT pr.idpractica) AS num_practicas, COUNT(DISTINCT nt.idtest) AS num_tests, COUNT(DISTINCT nep.idexamenP) AS examenes_aprobados
11 FROM personas p
12 JOIN estudiantes e ON p.idpersona = e.idpersona
13 JOIN practicas pr ON e.idpersona = pr.idestudiante
14 JOIN notas_tests nt ON e.idpersona = nt.idestudiante AND nt.aprobado = TRUE
15 JOIN notas_examenP nep ON e.idpersona = nep.idestudiante AND nep.aprobado = TRUE
16 GROUP BY p.idpersona;
```

Data Output Mensajes Notificaciones

Showing rows: 1 to 7 Page No: 1 of 1

	nombre character varying (50)	apellido1 character varying (60)	num_practicas bigint	num_tests bigint	examenes_aprobados bigint
1	Sebas	Vazques	2	1	1
2	Nerea	Santos	2	1	1
3	Lucia	Rodriguez	1	1	1
4	Diego	Vazquez	1	1	1
5	Ainhoa	Crespo	1	1	1
6	Carla	Souto	1	1	1
7	Eric	Vidal	1	1	1

3.- Al menos un outerjoin (que tenga sentido)

3. Estudiantes sin prácticas registradas. -> Lista los estudiantes que no tienen ninguna práctica registrada. El left join permite detectar la ausencia de registros en la tabla practicas.

```
20 SELECT DISTINCT(p.nombre), p.apellido1, pr.idpractica
21 FROM estudiantes e
22 JOIN personas p ON e.idpersona = p.idpersona
23 LEFT JOIN practicas pr ON e.idpersona = pr.idestudiante
24 WHERE pr.idpractica IS NULL;
```

Data Output Mensajes Notificaciones

	nombre character varying (50)	apellido1 character varying (60)	idpractica integer
1	Aron	Martinez	[null]
2	Javi	Pucelas	[null]
3	Pablo	Alvarez	[null]

4.- Al menos una subconsulta multifila

4. Mostrar los estudiantes que han aprobado algún test

```
29 SELECT p.idpersona,p.nombre,p.apellido1
30 FROM personas p
31 JOIN estudiantes e ON p.idpersona = e.idpersona
32 WHERE p.idpersona IN (
33     SELECT nt.idestudiante
34     FROM notas_tests nt
35     WHERE nt.aprobado = TRUE
36 );
```

Data Output Mensajes Notificaciones

      SQL

	idpersona [PK] integer	nombre character varying (50)	apellido1 character varying (60)
1	16	Sebas	Vazques
2	17	Aron	Martinez
3	20	Javi	Pucelas
4	21	Nerea	Santos
5	23	Lucia	Rodriguez
6	24	Pablo	Alvarez
7	26	Diego	Vazquez
8	27	Ainhua	Crespo
9	29	Carla	Souto
10	30	Eric	Vidal
11	16	Sebas	Vazques
12	17	Aron	Martinez
13	20	Javi	Pucelas
14	21	Nerea	Santos
15	23	Lucia	Rodriguez
16	24	Pablo	Alvarez
17	26	Diego	Vazquez
18	27	Ainhua	Crespo
19	29	Carla	Souto
20	30	Eric	Vidal

5.- Window Function (diferente a RANK)

5. Número acumulado de prácticas por estudiante. -> Muestra la duración acumulada de prácticas por estudiante a lo largo del tiempo. Usa SUM OVER como window function para calcular el total progresivo

```
40 SELECT Distinct(e.idpersona), p.nombre, p.apellido1, pr.fecha, SUM(pr.duracion_minutos) OVER (PARTITION BY e.idpersona ORDER BY pr.fecha) AS duracion_acumulada
41 FROM practicas pr
42 JOIN estudiantes e ON pr.idestudiante = e.idpersona
43 JOIN personas p ON e.idpersona = p.idpersona;
44
45
46 /*6. Relación entre tipo de examen práctico y número de faltas. -> Agrupa los exámenes prácticos por tipo y calcula el promedio de faltas, mostrando solo aquellos
47 SELECT ep.tipo, AVG(nep.num_faltas) AS promedio_faltas
```

Data Output Mensajes Notificaciones

Showing rows: 1 to 15 Page No: 1 of 1

	idpersona integer	nombre character varying (50)	apellido1 character varying (60)	fecha date	duracion_acumulada bigint
1	22	Adrian	Gomez	2025-12-07	140
2	23	Lucia	Rodriguez	2025-12-08	110
3	18	Marcos	Antepazo	2025-12-03	100
4	16	Sebas	Vazques	2025-12-02	230
5	30	Eric	Vidal	2025-12-15	110
6	21	Nerea	Santos	2025-12-06	210
7	21	Nerea	Santos	2025-12-05	90
8	25	Sara	Prieto	2025-12-10	230
9	29	Carla	Souto	2025-12-14	100
10	26	Diego	Vazquez	2025-12-11	120
11	19	Alejandro	Perez	2025-12-04	130
12	25	Sara	Prieto	2025-12-09	100
13	27	Ainhoa	Crespo	2025-12-12	90
14	28	Izan	Nogueira	2025-12-13	150
15	16	Sebas	Vazques	2025-12-01	120

6.- Agrupamientos y restricciones a nivel de grupo

6. Relación entre tipo de examen práctico y número de faltas. -> Agrupa los exámenes prácticos por tipo y calcula el promedio de faltas, mostrando solo aquellos tipos con más de 2 faltas promedio.

```
47 SELECT ep.tipo ,AVG(nep.num_faltas) AS promedio_faltas
48 FROM examenesP ep
49 JOIN notas_examenP nep ON ep.idexamenP = nep.idexamenP
50 GROUP BY ep.tipo
51 HAVING AVG(nep.num_faltas) > 2;
52
53
```

Data Output Mensajes Notificaciones

	tipo tipo_enum	promedio_faltas numeric
1	D	2.5000000000000000
2	A	5.0000000000000000
3	T	5.0000000000000000

Updates

```
--Marcar como aprobados (aprobado = TRUE) los tests de estudiantes que han hecho más de 1 práctica
UPDATE notas_tests nt
SET aprobado = TRUE
WHERE nt.idestudiante IN (
  SELECT pr.idestudiante
  FROM practicas pr
  GROUP BY pr.idestudiante
  HAVING COUNT(*) > 1
);
```

Antes

	idtest [PK] integer	idestudiante [PK] integer	aprobado boolean	fecha [PK] date	duracion_minutos integer	num_fallos integer
1	1	16	true	2025-12-...	20	1
2	2	17	true	2025-12-...	18	0
3	3	18	false	2025-12-...	25	7
4	4	19	false	2025-12-...	15	8
5	5	20	true	2025-12-...	30	2
6	6	21	true	2025-12-...	19	1
7	7	22	false	2025-12-...	14	5
8	8	23	true	2025-12-...	21	2
9	9	24	true	2025-12-...	10	0
10	10	25	false	2025-12-...	29	6
11	11	26	true	2025-12-...	17	1
12	12	27	true	2025-12-...	23	2
13	13	28	false	2025-12-...	13	4
14	14	29	true	2025-12-...	26	2
15	15	30	true	2025-12-...	20	1

Después

	idtest [PK] integer	idestudiante [PK] integer	aprobado boolean	fecha [PK] date	duracion_minutos integer	num_fallos integer
1	1	16	true	2025-12-...	20	1
2	2	17	true	2025-12-...	18	0
3	3	18	false	2025-12-...	25	7
4	4	19	false	2025-12-...	15	8
5	5	20	true	2025-12-...	30	2
6	6	21	true	2025-12-...	19	1
7	7	22	false	2025-12-...	14	5
8	8	23	true	2025-12-...	21	2
9	9	24	true	2025-12-...	10	0
10	10	25	true	2025-12-...	29	6
11	11	26	true	2025-12-...	17	1
12	12	27	true	2025-12-...	23	2
13	13	28	false	2025-12-...	13	4
14	14	29	true	2025-12-...	26	2
15	15	30	true	2025-12-...	20	1

```
--Actualizar teléfonos de personas que: viven en "Rua" y tienen teléfono NULL o mal formateado
UPDATE personas
SET telefono = REGEXP_REPLACE(COALESCE(telefono, '000000000'), '^[0-9]', '', 'g')
WHERE direccion ILIKE '%Rua%'
AND (telefono IS NULL OR telefono !~ '^[0-9 ]+$');
```

(no hay captura porque el resultado ocupa 2 páginas)

Deletes

```
--Eliminar notas de test con demasiados fallos (> 6)
DELETE FROM notas_tests
WHERE num_fallos > 6;
```

	idtest [PK] integer	idestudiante [PK] integer	aprobado boolean	fecha [PK] date	duracion_minutos integer	num_fallos integer		idtest [PK] integer	idestudiante [PK] integer	aprobado boolean	fecha [PK] date	duracion_minutos integer	num_fallos integer
1	2	17	true	2025-12-...	18	0	1	2	17	true	2025-12-...	18	0
2	3	18	false	2025-12-...	25	7	2	5	20	true	2025-12-...	30	2
3	4	19	false	2025-12-...	15	8	3	7	22	false	2025-12-...	14	5
4	5	20	true	2025-12-...	30	2	4	8	23	true	2025-12-...	21	2
5	7	22	false	2025-12-...	14	5	5	9	24	true	2025-12-...	10	0
6	8	23	true	2025-12-...	21	2	6	11	26	true	2025-12-...	17	1
7	9	24	true	2025-12-...	10	0	7	12	27	true	2025-12-...	23	2
8	11	26	true	2025-12-...	17	1	8	13	28	false	2025-12-...	13	4
9	12	27	true	2025-12-...	23	2	9	14	29	true	2025-12-...	26	2
10	13	28	false	2025-12-...	13	4	10	15	30	true	2025-12-...	20	1
11	14	29	true	2025-12-...	26	2	11	1	16	true	2025-12-...	20	1
12	15	30	true	2025-12-...	20	1	12	6	21	true	2025-12-...	19	1
13	1	16	true	2025-12-...	20	1	13	10	25	true	2025-12-...	29	6
14	6	21	true	2025-12-...	19	1							
15	10	25	true	2025-12-...	29	6							

```
--Eliminar prácticas de estudiantes que han suspendido examen práctico
DELETE FROM practicas pr
USING notas_examenP nep
WHERE pr.idestudiante = nep.idestudiante
AND nep.aprobado = FALSE;
```

	idpractica [PK] integer	idestudiante integer	idempleado_instructor integer	idvehiculo integer	fecha date	duracion_minutos integer	observaciones text
1	1	16	1	1	2025-12-...	60	Circuito básico
2	2	16	2	2	2025-12-...	55	Arranque en pendie...
3	3	18	3	3	2025-12-...	50	Rotondas
4	4	19	4	4	2025-12-...	65	Incorporaciones
5	5	21	5	5	2025-12-...	45	Señalización
6	6	21	6	6	2025-12-...	60	Maniobras
7	7	22	7	7	2025-12-...	70	Autovía
8	8	23	8	8	2025-12-...	55	Ciudad
9	9	25	9	9	2025-12-...	50	Estacionamiento
10	10	25	10	10	2025-12-...	65	Conducción nocturna
11	11	26	11	11	2025-12-...	60	Glorietas
12	12	27	12	12	2025-12-...	45	Cambio de carril
13	13	28	13	13	2025-12-...	75	Autopista
14	14	29	14	14	2025-12-...	50	Cruces
15	15	30	15	15	2025-12-...	55	Repaso general

	idpractica [PK] integer	idestudiante integer	idempleado_instructor integer	idvehiculo integer	fecha date	duracion_minutos integer	observaciones text
1	1	16	1	1	2025-12-...	60	Circuito básico
2	2	16	2	2	2025-12-...	55	Arranque en pendie...
3	5	21	5	5	2025-12-...	45	Señalización
4	6	21	6	6	2025-12-...	60	Maniobras
5	8	23	8	8	2025-12-...	55	Ciudad
6	11	26	11	11	2025-12-...	60	Glorietas
7	12	27	12	12	2025-12-...	45	Cambio de carril
8	14	29	14	14	2025-12-...	50	Cruces
9	15	30	15	15	2025-12-...	55	Repaso general

Transacciones

1.- Transacción monousuario con SELECT FOR UPDATE

```
29
30 --Bloquear prácticas de un estudiante mientras se revisan
31
32 BEGIN;
33
34 SELECT *
35 FROM practicas
36 WHERE idestudiante IN (
37     SELECT idpersona FROM personas WHERE nombre = 'Sebas'
38 )
39 FOR UPDATE;
40
41 --Añadimos mas tiempo a la practica
42 UPDATE practicas
43 SET duracion_minutos = duracion_minutos + 5
44 WHERE idestudiante IN (
45     SELECT idpersona FROM personas WHERE nombre = 'Sebas'
46 );
47
48 COMMIT;
```

Data Output Mensajes Notificaciones

	idpractica [PK] integer	idestudiante integer	idempleado_instructor integer	idvehiculo integer	fecha date	duracion_minutos integer	observaciones text
1	1	16	1	1	2025-12-...	65	Circuito básico
2	2	16	2	2	2025-12-...	60	Arranque en pendie...

2.- Transacción multiusuario con bloqueo compartido de tabla

```
--Permitir lecturas pero no modificaciones

--Usuario 1
BEGIN;
LOCK TABLE practicas IN SHARE MODE;

--Usuario 2
SELECT * FROM practicas; --permitido
UPDATE practicas SET duracion_minutos = 10; --bloqueado

--Usuario 1
ROLLBACK;
```

Desde el usuario 2 deja hacer el select pero cuando intentamos el update se pone en espera.

```
1 --Usuario 2
2 SELECT * FROM practicas; --permitido
3 UPDATE practicas SET duracion_minutos = 10; --bloqueado
4
```

Data Output Messages Notifications

	idpractica [PK] integer	idestudiante integer	idempleado_instructor integer	idvehiculo integer	fecha date	duracion_minutos integer	observaciones text
1	1	16	1	1	2025-12-...	60	Circuito básico
2	2	16	2	2	2025-12-...	55	Arranque en pendie...
3	3	18	3	3	2025-12-...	50	Rotondas
4	4	19	4	4	2025-12-...	65	Incorporaciones
5	5	21	5	5	2025-12-...	45	Señalización
6	6	21	6	6	2025-12-...	60	Maniobras
7	7	22	7	7	2025-12-...	70	Autovía
8	8	23	8	8	2025-12-...	55	Ciudad
9	9	25	9	9	2025-12-...	50	Estacionamiento
10	10	25	10	10	2025-12-...	65	Conducción nocturna
11	11	26	11	11	2025-12-...	60	Glorietas
12	12	27	12	12	2025-12-...	45	Cambio de carril
13	13	28	13	13	2025-12-...	75	Autopista
14	14	29	14	14	2025-12-...		
15	15	30	15	15	2025-12-...	55	Repaso general

Showing rows: 1 to

Waiting for the query to complete...

Una vez el usuario 1 hace el rollback se ejecuta el update.

Query Query History

```
1 --Usuario 2
2 SELECT * FROM practicas; --permitido
3 UPDATE practicas SET duracion_minutos = 10; --bloqueado
4
5
```

Data Output Messages Notifications

UPDATE 15

Query returned successfully in 26 secs 300 msec.

3.- Transacción multiusuario con bloqueo exclusivo de tabla

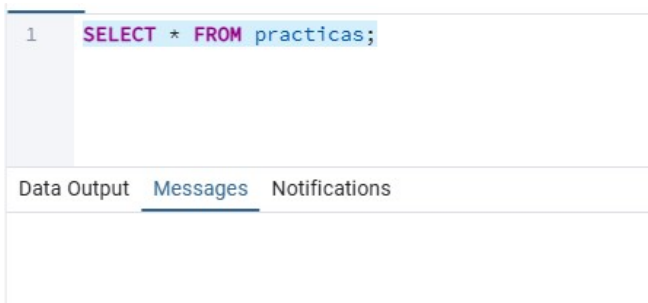
```
--Bloquear completamente la tabla

--Usuario 1
BEGIN;
LOCK TABLE practicas IN ACCESS EXCLUSIVE MODE;

--Usuario 2
SELECT * FROM practicas; --bloqueado

--Usuario 1
COMMIT;
```

El usuario 1 bloquea completamente la tabla por lo que desde el usuario 2 no nos permite hacer nada.



Una vez el usuario 1 acaba (hace el commit) se desbloquea la tabla y se ejecuta el código del usuario 2.

The screenshot shows the same SQL client interface, but now the 'Data Output' tab is selected and displays the results of the query `SELECT * FROM practicas;`. The results are shown in a table with 15 rows and 8 columns. Each cell in the table has a small edit icon (pencil) next to it.

	idpractica [PK] integer	idestudiante integer	idempleado_instructor integer	idvehiculo integer	fecha date	duracion_minutos integer	observaciones text
1	1	16	1	1	2025-12-...	10	Circuito básico
2	2	16	2	2	2025-12-...	10	Arranque en pendie...
3	3	18	3	3	2025-12-...	10	Rotondas
4	4	19	4	4	2025-12-...	10	Incorporaciones
5	5	21	5	5	2025-12-...	10	Señalización
6	6	21	6	6	2025-12-...	10	Maniobras
7	7	22	7	7	2025-12-...	10	Autovía
8	8	23	8	8	2025-12-...	10	Ciudad
9	9	25	9	9	2025-12-...	10	Estacionamiento
10	10	25	10	10	2025-12-...	10	Conducción nocturna
11	11	26	11	11	2025-12-...	10	Glorietas
12	12	27	12	12	2025-12-...	10	Cambio de carril
13	13	28	13	13	2025-12-...	10	Autopista
14	14	29	14	14	2025-12-...	10	Cruces
15	15	30	15	15	2025-12-...	10	Repaso general

Vistas

1.-

```
79  --Unir datos personales + vehículo
80
81  CREATE VIEW vista_estudiantes AS
82  SELECT DISTINCT(p.nombre),p.apellido1,v.marca,v.modelo
83  FROM estudiantes e
84  JOIN personas p ON e.idpersona = p.idpersona
85  JOIN vehiculos v ON e.idvehiculo = v.idvehiculo;
86
87
88  SELECT * FROM vista_estudiantes;
89  |
```

Data Output Messages Notifications

	nombre character varying (50) 🔒	apellido1 character varying (60) 🔒	marca character varying (30) 🔒	modelo character varying (30) 🔒
1	Adrian	Gomez	Renault	Clio
2	Aron	Martinez	Volkswagen	TDI
3	Diego	Vazquez	Ford	Focus
4	Sebas	Vazques	Nissan	GTI
5	Lucia	Rodriguez	Opel	Corsa
6	Eric	Vidal	Volkswagen	Golf
7	Marcos	Antepazo	Ford	Fiesta
8	Izan	Nogueira	Hyundai	i20
9	Pablo	Alvarez	Volkswagen	Polo
10	Ainhoa	Crespo	Opel	Astra
11	Carla	Souto	Toyota	Yaris
12	Sara	Prieto	Peugeot	208
13	Alejandro	Perez	Hyundai	Getz
14	Javi	Pucelas	Toyota	Corolla
15	Nerea	Santos	SEAT	Ibiza

2.-

```
91  --Ver resultados de tests con datos del alumno
92
93  CREATE VIEW vista_resultados_tests AS
94  SELECT DISTINCT (p.nombre),p.apellido1,nt.aprobado,nt.num_fallos
95  FROM notas_tests nt
96  JOIN personas p ON nt.idestudiante = p.idpersona;
97
98  SELECT *
99  FROM vista_resultados_tests
100 WHERE aprobado = TRUE;
101
102
```

Data Output Messages Notifications



	nombre character varying (50) 🔒	apellido1 character varying (60) 🔒	aprobado boolean 🔒	num_fallos integer 🔒
1	Nerea	Santos	true	1
2	Pablo	Alvarez	true	0
3	Lucia	Rodriguez	true	2
4	Diego	Vazquez	true	1
5	Javi	Pucelas	true	2
6	Sebas	Vazques	true	1
7	Carla	Souto	true	2
8	Eric	Vidal	true	1
9	Aron	Martinez	true	0
10	Ainhoa	Crespo	true	2

Funciones

1.-

¿Qué hace?

Recibe el ID de un estudiante y devuelve cuántos minutos de prácticas tiene acumulados en total.

¿Por qué es útil?

Los profesores pueden comprobar de un vistazo si un alumno tiene suficientes horas de conducción antes de presentarse al examen práctico, sin tener que hacer cuentas a mano.

```
CREATE OR REPLACE FUNCTION total_minutos_estudiante(p_idestudiante INT)
RETURNS INT AS $$
DECLARE
    v_nombre VARCHAR(60);
    v_total INT := 0;
BEGIN
    SELECT nombre INTO STRICT v_nombre
    FROM ONLY personas
    WHERE idpersona = p_idestudiante;

    SELECT COALESCE(SUM(duracion_minutos), 0) INTO v_total
    FROM practicas
    WHERE idestudiante = p_idestudiante;

    IF v_total = 0 THEN
        RAISE NOTICE '% no tiene prácticas registradas todavía.', v_nombre;
    ELSIF v_total < 200 THEN
        RAISE NOTICE '% lleva % min. Todavía necesita más prácticas.', v_nombre, v_total;
    ELSE
        RAISE NOTICE '% lleva % min. ¡Puede presentarse al examen práctico!', v_nombre, v_total;
    END IF;

    RETURN v_total;
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RAISE EXCEPTION 'No existe ningún estudiante con ID %.', p_idestudiante;
    WHEN OTHERS THEN
        RAISE EXCEPTION 'Error inesperado: %', SQLERRM;
END;
$$ LANGUAGE plpgsql;
```

33 `SELECT total_minutos_estudiante(16);`

Data Output		Messages	Notifications
	total_minutos_estudiante integer		
1	115		

34 `SELECT total_minutos_estudiante(17);`
 35 `SELECT total_minutos_estudiante(999);`

Data Output		Messages	Notifications
	total_minutos_estudiante integer		
1	0		

35 `SELECT total_minutos_estudiante(999);`

Data Output		Messages	Notifications
ERROR: No existe ningún estudiante con ID 999. CONTEXT: función PL/pgSQL total_minutos_estudiante(integer) en la línea 28 en RAISE			
SQL state: P0001			

2.-

¿Qué hace?

Devuelve una tabla con el listado de prácticas que ha impartido un instructor: el nombre del estudiante, la fecha, la duración y la matrícula del vehículo.

¿Por qué es útil?

Permite ver el historial de clases de cada profesor de forma clara, sin tener que escribir JOINS manualmente cada vez que se necesite esta información.


```

CREATE OR REPLACE FUNCTION practicas_del_instructor(p_idinstructor INT)
RETURNS TABLE (
    estudiante TEXT,
    fecha DATE,
    duracion_min INT,
    matricula VARCHAR
) AS $$
DECLARE
    cur_practicas CURSOR FOR
        SELECT
            (per.nombre || ' ' || per.apellido1) AS nombre_est,
            prac.fecha,
            prac.duracion_minutos,
            veh.matricula
        FROM practicas prac
        JOIN ONLY personas per ON per.idpersona = prac.idestudiante
        LEFT JOIN vehiculos veh ON veh.idvehiculo = prac.idvehiculo
        WHERE prac.idempleado_instructor = p_idinstructor
        ORDER BY prac.fecha;

    v_filas RECORD;
    v_nombre_inst VARCHAR(60);
    v_contador INT := 0;
BEGIN
    SELECT nombre INTO STRICT v_nombre_inst
    FROM ONLY personas
    WHERE idpersona = p_idinstructor;

    RAISE NOTICE 'Buscando prácticas de: %', v_nombre_inst;

    OPEN cur_practicas;
    LOOP
        FETCH cur_practicas INTO v_filas;
        EXIT WHEN NOT FOUND;

        v_contador := v_contador + 1;

        estudiante := v_filas.nombre_est;
        fecha := v_filas.fecha;
        duracion_min := v_filas.duracion_minutos;
        matricula := v_filas.matricula;

        RETURN NEXT;
    END LOOP;
    CLOSE cur_practicas;

    IF v_contador = 0 THEN
        RAISE NOTICE '% aún no tiene prácticas registradas.', v_nombre_inst;
    ELSE
        RAISE NOTICE 'Total de prácticas encontradas: %', v_contador;
    END IF;

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RAISE EXCEPTION 'No existe ningún instructor con ID %.', p_idinstructor;
    WHEN OTHERS THEN
        RAISE EXCEPTION 'Error inesperado: %', SQLERRM;
END;
$$ LANGUAGE plpgsql;

```

103

SELECT * FROM practicas_del_instructor(1);

Data Output

Messages

Notifications

+

📄

▼

📋

▼

🗑️

📦

⬇️

📈

SQL

	estudiante text 🔒	fecha date 🔒	duracion_min integer 🔒	matricula character varying 🔒
1	Sebas Vazqu...	2025-12-__	60	1111AAA

104

SELECT * FROM practicas_del_instructor(2);

Data Output

Messages

Notifications

SQL

	estudiante text	fecha date	duracion_min integer	matricula character varying
1	Sebas Vazqu...	2025-12-...	55	2222BBB

105	<code>SELECT * FROM practicas_del_instructor(999);</code>
Data Output	Messages
ERROR: No existe ningún instructor con ID 999.	
CONTEXT: función PL/pgSQL practicas_del_instructor(integer) en la línea 49 en RAISE	
SQL state: P0001	

Procedimiento

¿Qué hace?

Inserta una nueva práctica en la base de datos. El usuario pasa el DNI del estudiante y la matrícula del vehículo, y el procedimiento busca automáticamente sus IDs internos.

¿Por qué es útil?

Facilita el registro de prácticas sin necesidad de conocer los IDs de la base de datos. Además, valida que el estudiante y el vehículo existan antes de insertar, evitando errores manuales.

```
CREATE OR REPLACE PROCEDURE registrar_practica(
    p_dni          VARCHAR(9),
    p_matricula    VARCHAR(7),
    p_fecha        DATE,
    p_duracion     INT
)
AS $$
DECLARE
    v_idestudiante INT;
    v_idvehiculo   INT;
    v_nombre       VARCHAR(60);
BEGIN
    SELECT idpersona, nombre INTO STRICT v_idestudiante, v_nombre
    FROM ONLY personas
    WHERE dni = p_dni;

    SELECT idvehiculo INTO STRICT v_idvehiculo
    FROM vehiculos
    WHERE matricula = p_matricula;

    IF p_duracion <= 0 THEN
        RAISE EXCEPTION 'La duración debe ser mayor que 0 minutos.';
    END IF;

    INSERT INTO practicas (idestudiante, idvehiculo, fecha, duracion_minutos)
    VALUES (v_idestudiante, v_idvehiculo, p_fecha, p_duracion);

    RAISE NOTICE 'Práctica de % min registrada para % el %.', p_duracion, v_nombre, p_fecha;

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RAISE EXCEPTION
            'No se encontró al estudiante con DNI "%" o al vehículo con matrícula "%".',
            p_dni, p_matricula;
    WHEN TOO_MANY_ROWS THEN
        RAISE EXCEPTION
            'El DNI "%" aparece duplicado en la base de datos. Contacta con el administrador.',
            p_dni;
    WHEN OTHERS THEN
        RAISE EXCEPTION 'Error al registrar la práctica: %', SQLERRM;
END;
$$ LANGUAGE plpgsql;
```

154

155

156

157

158

```
CALL registrar_practica('33334444R', '2222BBB', '2026-01-10', 60);
SELECT idestudiante, fecha, duracion_minutos
FROM practicas
WHERE idestudiante = 17;
```

Data Output

Messages

Notifications

≡

📄

▼

📋

▼

🗑️

🔄

⬇️

📈

SQL

	idestudiante integer	fecha date	duracion_minutos integer
1	17	2026-01-10	60

160	CALL registrar_practica('99999999Z', '1111AAA', '2026-01-11', 45);
161	
Data Output Messages Notifications	
ERROR: No se encontró al estudiante con DNI "99999999Z" o al vehículo con matrícula "1111AAA". CONTEXT: función PL/pgSQL registrar_practica(character varying,character varying,date,integer) en la línea 26 en RAISE	

162	CALL registrar_practica('33334444R', 'ZZZZZZZ', '2026-01-12', 50);
Data Output Messages Notifications	
ERROR: No se encontró al estudiante con DNI "33334444R" o al vehículo con matrícula "ZZZZZZZ". CONTEXT: función PL/pgSQL registrar_practica(character varying,character varying,date,integer) en la línea 26 en RAISE SQL state: P0001	

Triggers

1.-

¿Qué hace?

Comprueba que, si se indica un instructor (idempleado_instructor NOT NULL), ese ID pertenezca a alguien que está en la tabla empleados. Si no es así, cancela la inserción lanzando una excepción.

¿Por qué es útil?

La clave foránea de practicas solo verifica que el ID exista en personas, pero no que sea un empleado. Este trigger añade esa validación extra, evitando que se registre como instructor a un estudiante o a alguien externo que no trabaje en la autoescuela.

```
CREATE OR REPLACE FUNCTION fn_validar_instructor()
RETURNS TRIGGER AS $$
DECLARE
    v_es_empleado INT;
BEGIN
    IF NEW.idempleado_instructor IS NULL THEN
        RETURN NEW;
    END IF;

    SELECT COUNT(*) INTO v_es_empleado
    FROM empleados
    WHERE idpersona = NEW.idempleado_instructor;

    IF v_es_empleado = 0 THEN
        RAISE EXCEPTION
            'La persona con ID % no es un empleado de la autoescuela.',
            NEW.idempleado_instructor;
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_validar_instructor
BEFORE INSERT ON practicas
FOR EACH ROW
EXECUTE FUNCTION fn_validar_instructor();
```

203

204

205

INSERT INTO practicas (idestudiante, idempleado_instructor, idvehiculo, fecha, duracion_minutos)

VALUES (20, 3, 3, '2026-01-15', 55);

SELECT * FROM practicas WHERE idestudiante = 20 AND fecha = '2026-01-15';

Data Output

Messages

Notifications

≡

+

📄

▼

📁

▼

🗑️

🔄

📶

📶

📶

📶

SQL

Showing rows: 1 to 1

	idpractica [PK] integer	idestudiante integer	idempleado_instructor integer	idvehiculo integer	fecha date	duracion_minutos integer	observaciones text	
1	17	20	3	3	2026-01-15	55	[null]	

207	INSERT INTO practicas (idestudiante, idempleado_instructor, idvehiculo, fecha, duracion_minutos)
208	VALUES (18, 16, 4, '2026-01-16', 60);
209	
Data Output Messages Notifications	
ERROR: La persona con ID 16 no es un empleado de la autoescuela. CONTEXT: función PL/pgSQL fn_validar_instructor() en la línea 17 en RAISE SQL state: P0001	

210

INSERT INTO practicas (idestudiante, idempleado_instructor, idvehiculo, fecha, duracion_minutos)

211

VALUES (24, NULL, 5, '2026-01-17', 45);

212

SELECT * FROM practicas WHERE idestudiante = 24 AND fecha = '2026-01-17';

213

Data Output

Messages

Notifications

SQL

Showing rows: 1

	Idpractica [PK] integer	idestudiante integer	idempleado_instructor integer	idvehiculo integer	fecha date	duracion_minutos integer	observaciones text
1	19	24	[null]	5	2026-01-17	45	[null]

2.-

¿Qué hace?

- En un INSERT: guarda en log_practicas que se añadió una nueva práctica y cuántos minutos tiene.
- En un UPDATE: solo guarda un registro si la duración cambió (compara OLD.duracion_minutos con NEW.duracion_minutos).

Si solo cambiaron otros campos (p. ej. observaciones), no escribe nada en el log.

¿Por qué es útil?

Mantiene un historial de cambios en las prácticas. Así se puede saber en cualquier momento cuándo se añadió una clase o cuándo alguien modificó su duración.

Tabla creada para el trigger

```
--Tabla para el trigger 2
CREATE TABLE IF NOT EXISTS log_practicas (
  idlog          SERIAL PRIMARY KEY,
  idestudiante   INT,
  fecha_cambio   TIMESTAMP DEFAULT NOW(),
  tipo_operacion VARCHAR(10),
  duracion_anterior INT,
  duracion_nueva INT,
  descripcion     TEXT
);
```

```

229 CREATE OR REPLACE FUNCTION fn_log_practicas()
230 RETURNS TRIGGER AS $$
231 BEGIN
232     IF TG_OP = 'INSERT' THEN
233         INSERT INTO log_practicas (idestudiante, tipo_operacion, duracion_anterior, duracion_nueva, descripcion)
234         VALUES (
235             NEW.idestudiante,
236             'INSERT',
237             NULL,
238             NEW.duracion_minutos,
239             'Nueva práctica añadida. Fecha: ' || NEW.fecha
240         );
241
242     ELSIF TG_OP = 'UPDATE' THEN
243         IF OLD.duracion_minutos <> NEW.duracion_minutos THEN
244             INSERT INTO log_practicas (idestudiante, tipo_operacion, duracion_anterior, duracion_nueva, descripcion)
245             VALUES (
246                 NEW.idestudiante,
247                 'UPDATE',
248                 OLD.duracion_minutos,
249                 NEW.duracion_minutos,
250                 'Duración cambiada de ' || OLD.duracion_minutos || ' a ' || NEW.duracion_minutos || ' min'
251             );
252         END IF;
253     END IF;
254
255     RETURN NULL;
256 END;
257 $$ LANGUAGE plpgsql;
258
259 CREATE TRIGGER trg_log_practicas
260 AFTER INSERT OR UPDATE ON practicas
261 FOR EACH ROW
262 EXECUTE FUNCTION fn_log_practicas();
263

```

```
265 INSERT INTO practicas (idestudiante, idvehiculo, fecha, duracion_minutos)
266 VALUES (26, 7, '2026-02-01', 70);
267 SELECT * FROM log_practicas ORDER BY idlog DESC LIMIT 3;
268
```

	Idiog [PK] integer	Idestudiante integer	fecha_cambio timestamp without time zone	tipo_operacion character varying (10)	duracion_anterior integer	duracion_nueva integer	descripcion text
1	1	26	2026-06-06 17:57:37.750249	INSERT	[null]	70	Nueva práctica añadida. Fecha: 2026-02...


```

269 UPDATE practicas
270 SET duracion_minutos = 90
271 WHERE idestudiante = 26 AND fecha = '2026-02-01';
272 SELECT * FROM log_practicas ORDER BY idlog DESC LIMIT 3;

```

	idlog [PK] integer	idestudiante integer	fecha_cambio timestamp without time zone	tipo_operacion character varying (10)	duracion_anterior integer	duracion_nueva integer	descripcion text
1	2	26	2026-06-06 17:58:55.668525	UPDATE	70	90	Duración cambiada de 70 a 90 min
2	1	26	2026-06-06 17:57:37.750249	INSERT	[null]	70	Nueva práctica añadida. Fecha: 2026-02...

```

274 UPDATE practicas
275 SET observaciones = 'Práctica revisada'
276 WHERE idestudiante = 26 AND fecha = '2026-02-01';
277 SELECT * FROM log_practicas ORDER BY idlog DESC LIMIT 3;
278

```

	idlog [PK] integer	idestudiante integer	fecha_cambio timestamp without time zone	tipo_operacion character varying (10)	duracion_anterior integer	duracion_nueva integer	descripcion text
1	2	26	2026-06-06 17:58:55.668525	UPDATE	70	90	Duración cambiada de 70 a 90 min
2	1	26	2026-06-06 17:57:37.750249	INSERT	[null]	70	Nueva práctica añadida. Fecha: 2026-02...

Backup

`pg_dump -h localhost -U postgres -p 5432 -F p -v -f backup_hermemehildo.sql Hermemehildo`

Explicación de los parámetros:

- **-h localhost -U postgres:** Define el host y el superusuario para asegurar permisos sobre funciones y triggers.
- **-F p:** Formato "plain text" (.sql). Es ideal para ver el código SQL de los procedimientos y triggers.
- **-v:** Modo "verbose", para supervisar el progreso de la exportación de tablas y funciones.
- **-f backup_hermemehildo.sql:** Nombre del archivo de salida.
- **Hermemehildo:** nombre de la base de datos en pg_admin

Para comprobar la integridad, se puede restaurar en una base de datos limpia con :

`psql -U postgres -d bd_nueva -f backup_hermemehildo.sql`

bd_nueva tiene que estar ya creada.